Khmer ASR presentation rehearsal script

For the 13-slide deck updated 20 October 2026. Target speaking time: about 10 minutes; the questions below are for discussion, not part of the timed talk.

Slide 1

Good morning. I am Khemrak Pasey. My project is Khmer automatic speech recognition: converting spoken Khmer into written Khmer. I compare three checkpoints that I adapted from publicly pretrained models. I will explain why I chose them, how I trained them, and what their shared held-out test results do and do not show.

Slide 2

The user problem is getting usable Khmer text from recorded speech. People need transcripts for captions, interviews, and notes, but manual transcription requires repeated listening, typing, and proofreading. My project evaluates whether an adapted speech-recognition model can produce a more accurate first transcript of a short Khmer utterance. I compare three saved approaches by counting remaining character and word errors on held-out utterances. I measure transcription error, not the amount of human time saved.

Slide 3

The data source is FLEURS Khmer. An utterance is a short audio recording with a reference transcript. Whisper and MMS trained on the same 531 selected utterances. All three approaches used the same 124 utterances for validation and the same 114 held-out utterances for the final test. The Qwen adapter used 16,720 effective training examples. That count includes repeated FLEURS training examples and additional public Khmer speech, so it is not a count of unique utterances. This makes the comparison descriptive rather than a controlled architecture experiment.

Slide 4

Character error rate and word error rate use the same edit-distance formula: substitutions plus deletions plus insertions, divided by reference units, times 100 percent. For CER, the units are characters after Unicode normalization and removal of whitespace and zero-width marks. For WER, the units are Khmer word tokens found by ICU on both reference and prediction. We sum errors over the whole corpus before dividing. Lower is better; WER can exceed 100 percent if there are many insertions.

Slide 5

These are not three untouched Hugging Face models, and I did not train them from random initialization. Approach 1 started with openai/whisper-tiny and I fully fine-tuned its weights on 531 FLEURS utterances. Approach 2 started with facebook/mms-1b-all and I trained its top four speech encoder layers plus the CTC output on the same 531 utterances; validation selected checkpoint step 300. Approach 3 started with the publisher-trained seanghay Qwen3-ASR 0.6B Khmer checkpoint. I trained a LoRA adapter on a larger corpus with 16,720 effective examples; validation selected epoch 1. The base checkpoints and training datasets differ.

Slide 6

I chose the three approaches to cover different model families and training methods. Whisper-Tiny is a small encoder-decoder baseline trained with full fine-tuning. MMS-1B is a much larger multilingual acoustic encoder with CTC output, and I tuned only its upper layers and output. Qwen3-ASR is a Khmer-adapted generative starting point for a parameter-efficient LoRA experiment. This makes the comparison useful for exploring practical approaches. Because Qwen had much more training data and a Khmer-specific starting checkpoint, the test scores cannot isolate architecture as the cause of the difference.

Slide 7

My process had four stages. First, I prepared short Khmer audio utterances and reference transcripts. Second, I started from three existing pretrained checkpoints and adapted each differently: full fine-tuning for Whisper, the top four encoder layers and CTC head for MMS, and a LoRA adapter for Qwen. Third, I chose checkpoints using the separate 124-utterance validation set. Finally, I scored the selected checkpoints on the same 114 held-out test utterances. All runs used a Tesla T4 GPU. Whisper trained for three epochs; its complete recorded training call took 521 seconds, about 8.7 minutes. MMS trained for 15 epochs, with an interruption and resume; about 1.6 hours is an estimate from the last segment's step rate, because the exact total wall time was not saved. Qwen trained for three epochs on 16,720 effective training entries. Epoch 1 was selected by validation WER. About 6.3 hours of active training is an estimate from a logged segment; setup, validation, and interruptions may add time. I would not present the two estimates as measured totals.

Slide 8

On the same 114 held-out utterances, Whisper-Tiny scored 86.76 percent CER and 106.27 percent WER. MMS-1B scored 15.84 percent CER and 67.16 percent WER. Qwen3-ASR LoRA scored 13.71 percent CER and 33.40 percent WER. Qwen has the lowest observed error. The training examples column is essential: Qwen used 16,720 effective examples, while Whisper and MMS used 531. The Qwen base checkpoint was also already adapted for Khmer before my LoRA training. These are shared-test measurements, not a controlled comparison of architectures.

Slide 9

The Qwen adapter ran for three epochs. I selected epoch 1 because it had the lowest WER on the 124 validation utterances: 12.42 percent CER and 27.99 percent WER. That validation WER missed the planned below-25-percent goal. After selection, I evaluated the saved adapter once on 114 held-out test utterances, obtaining 13.71 percent CER and 33.40 percent WER. Validation and test use different utterances, so the difference is descriptive. Only test values appear in the three-model result table.

Slide 10

The table needs interpretation. Whisper made 2,644 word edits across 2,488 reference words, so WER exceeds 100 percent and this saved checkpoint is not reliable on the shared test. MMS improved greatly at the character level but still had 67.16 percent word error rate. Its 1,671 word edits include 605 insertions; character accuracy does not guarantee the right word tokens under ICU segmentation. Qwen made 831 word edits, 840 fewer than MMS, but it used far more effective training examples and started from a Khmer-adapted published checkpoint. I cannot say architecture or LoRA alone caused the difference. These are text edit patterns; I have not verified acoustic causes by listening.

Slide 11

A closer saved analysis of MMS finds 23.75 percent character error on 26 test references containing Latin letters, compared with 13.03 percent on the other 88. Names and mixed-script material may be difficult, but this association does not prove an acoustic cause. The Qwen comparison is confounded by 31.5 times as many effective training examples and a publisher-trained Khmer base. The 114 FLEURS test utterances are also a limited sample; everyday recordings could differ. Finally, ICU Khmer segmentation is a documented scoring choice and may affect WER. The next work is to listen to difficult utterances and classify errors, then run a comparison with matched training data if architecture is the question.

Live demo cue (after Slide 11, if time allows)

I will now demonstrate the selected Qwen adapter on one short Khmer utterance. I prepared the Colab runtime and checked that the exact epoch-1 adapter loads before this presentation. I will upload an audio file and run inference live. This example shows how the system produces a transcript; it is not part of the 114-utterance test and does not change the reported CER or WER. I will play the audio, then show the generated text and the time taken. If the network or Colab runtime is unavailable, I will use the same saved adapter in my local app.

Slide 12

The main result is that Qwen3-ASR achieved the lowest observed error in the chosen lineup on the fixed 114-utterance test: 13.71 percent CER and 33.40 percent ICU Khmer WER. The Qwen validation goal of below 25 percent WER remains unmet. The unequal training data and different starting checkpoints prevent a claim that Qwen architecture alone is better. The strongest next step is a listening-based error audit followed by a matched-data experiment. Thank you.

Slide 13

The data source is FLEURS. The starting model families are described in the original Whisper and MMS papers and the Qwen Khmer model page. The project repository contains saved prediction pairs, scoring policy, training configuration, and result summaries. AI assistance supported review, analysis, code, documentation, and this presentation; I am responsible for verifying and explaining the submitted work.